

问题与机制：把上下文当作工作台

一句话结论：MemGPT 让模型用工具管理记忆，把当前需要的信息搬进有限的上下文。它面向长期聊天和跨文档分析：对话不断增长，重要事实可能离开窗口；把历史压成一份摘要，又可能丢掉后来才需要的细节。

论文借鉴操作系统的虚拟内存，区分本轮推理可见的主上下文与外部存储。这里的“无限上下文”是持续存取历史的能力，模型单次接收的 token 仍然有限。**信息已经保存，不代表模型此刻能看见；检索结果也会占用窗口。**



三种记忆各司其职。 Core 是系统提示词内可编辑的短文本，常放角色设定、用户偏好和重要事实；代码中的 `persona`、`human` 对应这块工作记忆。Recall 自动保存交互消息，旧消息移出窗口后仍可查询原文。Archival 保存选定知识和导入资料，内容需要检索后才能进入推理。

以“我上次说想去哪里旅行？”为例：模型发现当前窗口缺少答案，调用历史搜索；程序把“想去京都”的相关记录作为工具结果加入窗口；模型在下一次调用中读到它，再发送回答。信息不足时，可以改查询或继续取结果。

贡献集中在三个连接点：**分层存储**保留长期信息，**模型自主读写**决定此刻需要什么，**函数连续调用**让一次用户请求包含多次检索与推理。`request_heartbeat=true` 请求工具结束后继续运行模型；程序仍负责执行函数、维护消息与判断是否继续。模型选择策略，运行时承担具体的数据操作。

工具与实证：它具体能做什么

0.3.1 默认聊天与文档预设启用以下 **8 个工具**。六个记忆工具的 schema 还包含 `request_heartbeat`，供模型请求下一轮推理。

类别	工具	用途
对外输出	<code>send_message</code>	展示用户可见的回答
定时调度	<code>pause_heartbeats</code>	暂停定时唤醒，最长 360 分钟
Core	<code>core_memory_append</code> 、 <code>core_memory_replace</code>	追加或精确替换便签文本
Recall	<code>conversation_search</code> 、 <code>conversation_search_date</code>	按文本或日期搜索历史
Archival	<code>archival_memory_insert</code> 、 <code>archival_memory_search</code>	写入知识或做语义检索

Core 已经在上下文中，因此无需搜索工具；Recall 由程序记录，因此无需让模型逐条写入。`send_message` 负责展示消息，外层循环再根据预警、工具错误和 heartbeat 决定是否继续，它本身不是强制结束指令。

窗口满了怎么办？ 论文用约 70% 预警、满载刷新、驱逐约 50% 作为示例。核对的 0.3.1 则在一次调用返回的 `usage.total_tokens` 超过窗口 **75%** 时发出预警，让模型考虑保存要点；识别到上下文溢出错误后，程序另行调用模型，把一段旧消息替换成摘要并重试。Core 与近期消息保留，Recall 原文仍在。预警、主动存档和摘要压缩是三个动作。

最清晰的实证来自深度记忆检索（DMR）。 任务要求回答明确指向先前聊天细节的问题。MSC 每组聊天包含五个会话，DMR 再为同一对角色构造第六个会话中的单个问答；固定上下文基线只能看有损历史摘要，MemGPT 可以搜索完整历史。下表均为同底模比较，Accuracy 由 LLM judge 对照标准答案判定。

底层模型	摘要基线	加入 MemGPT
GPT-3.5 Turbo	38.7%	66.9%
GPT-4	32.1%	92.5%
GPT-4 Turbo	35.3%	93.4%

结果支持“保留原文并按需找回”在 DMR 中的价值。它比较的是两套历史访问方案，不能直接解释成所有任务的通用增益。对比同时改变了可访问历史的信息量与访问方式，仅凭该表无法分离各组件的贡献。迁移到业务时，需要另设相同数据范围和调用预算的基线，观察是否仍有收益。

论文还考察文档问答和嵌套键值检索：多次查询有助于补充证据与完成多跳查找，但效果随底模变化；GPT-3.5 的函数调用能力构成明显瓶颈。

阅读判断：什么时候值得采用

我的判断：MemGPT 最有启发的是把记忆管理变成可观察、可调度的工具行为。 长期助手可以修改一条用户事实，检索旧对话的证据，再据此回答；每一步都有明确的输入和结果。这给记忆评估提供了切入点：分别检查存得是否正确、查得是否相关、读到证据后是否答对。

它适合长期个性化助手、持续积累的项目助理，以及需要反复追查资料的分析任务。选型时应先确认两件事：重要信息确实频繁跨出窗口，任务也允许额外检索与模型调用。对于历史很短、延迟要求严格或一次检索即可回答的任务，收益需要通过实际工作负载衡量。

局限需要沿着流程看。 模型可能漏存要点、写下错误事实或未更新旧偏好；查询措辞可能找不到目标，检索后也可能过早停止。论文文档问答只采样了 50 个问题，并观察到模型常在遍历完候选前停止翻页。摘要仍然有损，工具链增加耗时与调用成本；扩大外部存储不会自动改善推理能力。

实验范围也有限：DMR 问答由另一 LLM 构造，准确率依赖 LLM judge，底模是当时的特定版本。论文中的 GPT-4 指 `gpt-4-0613`，Turbo 指 `gpt-4-1106-preview`。将结果用于今天的系统设计时，应以自己的数据重新评估，并分别统计正确率、检索次数、token 与延迟。

五问速记

1. **Core 在哪里？** 在每次调用的系统提示词中，是可编辑文本。
2. **谁自动保存对话？** 程序写入 Recall；Archival 保存主动写入或导入的知识。
3. **如何连续检索？** 工具请求 heartbeat，结果回到上下文，再运行模型。
4. **75% 会自动清空记忆吗？** 不会；这是所查版本的预警，摘要另由溢出路径触发。
5. **92.5% 说明什么？** 论文 GPT-4+MemGPT 在 DMR 上的准确率，不能推广到全部任务。

来源与核对边界

论文：Packer 等，MemGPT，arXiv v2，方法、实验与 DMR 表；原文署名属于原作者，许可为 CC BY 4.0。中文翻译、读书报告与教学图由 Clearer Notes 整理。

实现：官方 0.3.1 固定提交，核对 `agent.py`、`memory.py`、`constants.py`、`main.py` 与 `functions/`。这是历史源码静态核对；分页接口存在兼容性瑕疵，不能据此保证该版本可直接运行。本报告未运行模型 API，未独立复现实验。